

Bachelorarbeit

Entwurf einer erweiterbaren Softwarearchitektur im Inbetriebnahmemanagement unter Nutzung des IFC-Standards

vorgelegt von:	Jan Händl
Fakultät:	Informatik/Mathematik
Studiengang:	Medieninformatik
Ort:	HTW Dresden
Matrikelnummer:	50290
Erstgutachter:	Prof. Dr. Tim Pidun
Zweitgutachter:	Dipl.-Inf. Claus Engelhardt
Zeitraum:	19.05.2026 - 14.07.2026
Abgabedatum:	14.07.2026

Abstract

Hier kommt noch ein abstract rein ()

Inhaltsverzeichnis

Abkürzungsverzeichnis

Glossar

ACID Ein Akronym für die vier geforderten Eigenschaften von Datenbanktransaktionen zur Sicherstellung der Datenintegrität: **A**tomicity (Atomarität), **C**onsistency (Konsistenz), **I**solation (Isolation) und **D**urability (Dauerhaftigkeit). [5](#)

EXPRESS EXPRESS ist ein Standard für eine generische Datenmodellierungssprache für Produktdaten. EXPRESS ist im ISO-Standard für den Austausch von Produktmodelldaten STEP (ISO 10303) formalisiert und als ISO 10303-11 standardisiert.. [4](#)

Rest-API Ein Architekturstil für verteilte Systeme, der häufig für die Entwicklung von Web-APIs genutzt wird (Representational State Transfer). . [6](#)

SPA Eine Single-Page-Application (deutsch: Einelseiten-Anwendung) ist eine Webapplikation, die aus einem einzigen HTML-Dokument besteht. Anstatt bei jeder Nutzerinteraktion die komplette Seite vom Server neu zu laden, werden Inhalte dynamisch via JavaScript (z. B. durch *React*) nachgeladen und manipuliert. Dies ermöglicht eine flüssige Benutzererfahrung, die der einer nativen Desktop-Anwendung ähnelt. Im Kontext des Inbetriebnahme-Managements ist dies die Voraussetzung für die *Offline-Fähigkeit*, da die Anwendungslogik bereits im Browser geladen ist und auch ohne aktive Netzverbindung auf Benutzereingaben reagieren kann. [7](#)

STEP Standard for the Exchange of Product Model Data (normiert nach ISO 10303). Ein plattformunabhängiges, standardisiertes Datenformat zum Austausch von 3D-Produktdaten und Bauwerksmodellen zwischen heterogenen CAX- und BIM-Systemen. Im Kontext von *Building Information Modeling* (BIM) bildet die physikalische Textdatei nach ISO 10303-21 (*Clear Text Encoding*) das fundamentale Austauschformat für klassische ifc-Dateien. . [9](#)

Webpack Ein moderner Open-Source-Modul-Bundler für JavaScript-Anwendungen. Webpack analysiert den Modul-Graphen einer Webapplikation, löst Abhängigkeiten auf und bündelt diese in optimierte statische Assets. In der vorliegenden Arbeit dient Webpack als technische Basis für die Implementierung der Micro-Frontend-

Architektur mittels *Module Federation*, welches das dynamische Laden von funktionalen Add-Ons zur Laufzeit ermöglicht. [7](#)

Abbildungsverzeichnis

1 Einleitung

1.1 Problemstellung

Die Baubranche stellt eine der tragenden Säulen der deutschen Wirtschaft dar, sieht sich jedoch im Vergleich zu anderen Industriezweigen mit einer signifikanten Digitalisierungslücke konfrontiert (?). Während in der Planungsphase durch Methoden wie Building Information Modeling (BIM) bereits digitale Fortschritte erzielt werden, hinkt insbesondere die Ausführungsphase auf der Baustelle deutlich hinterher. Dieser Rückstand führt in der Praxis zu einer Vielzahl von Ineffizienzen, die den Projekterfolg bei Großbauvorhaben gefährden.

Ein zentrales Hemmnis ist die mangelnde Interkonnektivität der eingesetzten Softwaresysteme. Da einheitliche Standards und Schnittstellen oft fehlen, entstehen isolierte Datensilos. Dies führt regelmäßig zum sogenannten „Excel-Fallback“ (?): Um den Informationsfluss zwischen den unterschiedlichen Systemen und verschiedenen Akteuren aufrechtzuerhalten, werden Daten manuell in Tabellenkalkulationen übertragen. Dieser Prozess ist nicht nur fehleranfällig, sondern verhindert auch eine revisionssichere Dokumentation und Echtzeit-Auswertungen.

Zusätzlich verschärft das Problem des Vendor-Locks die Situation (?). Unternehmen sind oft an geschlossene Ökosysteme einzelner Softwareanbieter gebunden, was die Erweiterbarkeit um spezialisierte Funktionen oder die Integration innovativer Drittanbieter-Lösungen massiv erschwert. Bestehende Anwendungen agieren häufig als Insellösungen, die den spezifischen Anforderungen komplexer Bauprozesse, wie einem durchgängigen Inbetriebnahmemanagement, nicht gerecht werden

Erschwert wird der digitale Einsatz vor Ort durch die oft mangelhafte Infrastruktur (?). Auf Baustellen ist eine stabile Mobilfunkabdeckung oder Breitbandanbindung keine Selbstverständlichkeit. Viele aktuelle Cloud-Lösungen setzen jedoch eine permanente Internetverbindung voraus und stoßen bei instabilem Netz an ihre Grenzen. Ohne robuste Offline-First-Strategien führt dies zu Datenverlusten oder Unterbrechungen im digitalen Workflow, was die Akzeptanz digitaler Werkzeuge beim Personal vor Ort weiter sinken lässt.

1.2 Forschungslücke

Trotz der Verfügbarkeit diverser Softwarelösungen im BIM-Kontext, primär in den Bereichen des digitalen Bautagebuchs und der Mängelerfassung, weisen diese oftmals Defizite hinsichtlich ihrer Flexibilität und Adaptierbarkeit an spezifische Projektanforderungen auf. Zur Lösung dieser Problematik ist eine modulare Architektur erforderlich, die es ermöglicht, spezialisierte Anwendungen flexibel als Add-ons zu integrieren, sodass diese nahtlos auf bestehende Daten zugreifen können. Um dies zu ermöglichen, bedarf es einer standardisierten und transparenten Schnittstellen um einen problemlosen Informationsaustausch zu gewährleisten. Es existieren bereits Forschungen und etablierte Ansätze für graphbasierte Domänenmodelle, sowie die Nutzung von GraphQL als Vermittlungsschicht, um die komplexen Strukturen des IFC Standards abzubilden. Arbeiten wie (?) zeigen bereits, dass die Überführung von IFC-Daten in ein GraphQL Schemata grundlegend machbar ist, jedoch eine exakte 1:1 Abbildungen an ihre Grenzen hinsichtlich der Komplexität stößt. Trotzdem wird deutlich, dass eine solche Überführung Vorteile in Form von Effizienz und Entwicklerfreundlichkeit mit sich bringt.

Es fehlt jedoch an einer Verknüpfung dieser Modelle mit einer modularen Architektur als Frontend, mit welcher die beschriebenen Probleme aus , gezielt angegangen werden können.

1.3 Forschungsfragen

Um die oben genannten Zielsetzungen methodisch zu untersuchen, konzentriert sich diese Arbeit auf die Beantwortung der folgenden zwei Kernfragen:

FF1: Wie lässt sich eine Graph-basierte Datenstruktur auf Basis des IFC-Standards gestalten, um die komplexen funktionalen Abhängigkeiten der TGA und MSR effizient abzubilden und als Grundlage für ein erweiterbares Inbetriebnahmemanagement zu nutzen?

FF2: Wie lässt sich eine modulare Multi-Tenant-Architektur für Inbetriebnahmemanagement-Software gestalten, welche funktionale Erweiterungen ermöglicht?

1.4 Zielsetzung

Das primäre Ziel dieser Arbeit ist die Konzeption einer technologischen Architektur, die den systemimmanenten Vendor-Lock und die Problematik proprietärer Insellösungen durch Transparenz überwindet. Im Zentrum steht dabei die Nutzung des **IFC! (IFC!)**-Standards als universelle Sprache für Bauwerksdaten. Um die komplexen Beziehungen zwischen Bauteilen, Gewerken und Prozessen performant abzubilden, wird das System auf einer Graphdatenbank aufgesetzt. Im Gegensatz zu starren relationalen Tabellen ermöglicht die Graphentechnologie eine flexible Verknüpfung der Datenquellen und bildet so das Rückgrat für die Interkonnektivität zwischen den Projektbeteiligten. Ergänzt wird dieser datenzentrierte Ansatz durch eine Add-On-fähige Software-Architektur. Diese erlaubt es, spezifische Anforderungen für das Inbetriebnahmemanagement als modulare Erweiterungen zu implementieren, ohne den Software-Kern modifizieren zu müssen. Durch die Kombination aus standardisiertem Datenaustausch (**IFC!**) und modularer Erweiterbarkeit entsteht ein Ökosystem, das nicht nur den Datenaustausch zwischen verschiedenen Gewerken sicherstellt, sondern Unternehmen die Souveränität über ihre digitalen Prozesse zurückgibt und eine zukunftsichere Skalierbarkeit in Großprojekten ermöglicht.

1.5 Methodik

Um die beschriebenen Zielsetzung zu erreichen wird diese Arbeit in drei Teilschritte unterteilt. Zuerst soll durch Literaturrecherche und prototypischen Tests ein Domänenmodell mit der Graphdatenbank *Neo4j* und dem Framework *GraphQL* als Vermittlerinstanz auf seine Vorteile und Machbarkeit untersucht werden. Als nächster Schritt soll eine modulare Softwarearchitektur, aufbauend auf dem Konzept der Microfrontends, entworfen werden. Die technische Umsetzung soll mit dem Build-Tool *Vite* und dem Architekturmuster *Module Federation* erfolgen. Das finale Ziel dieses Schrittes ist ein Prototyp als Proof of Concept. Das Zusammenspiel dieser beiden Technologien bietet eine moderne und entwicklerfreundliche Möglichkeit, Microfrontends als Webapplikationen herauszuführen. Auf diesen beiden Lösungsansätzen aufbauend soll als letzter Schritt die Zusammenführung beider Teilsysteme erfolgen. Hierbei wird der Prototyp der modularen Softwarearchitektur über die GraphQL Schnittstelle an die Graphdatenbank angebunden.

2 Grundlagen

2.1 IFC-Standard

Der IFC Standard (Industry Foundation Classes) ist eine offene, internationale Spezifikation zur Beschreibung von Bauelementen und deren Beziehungen zueinander. Er wird seit 1994 von der Organisation buildingSMART im **EXPRESS** Schema entwickelt, um die Digitalisierung auf Baustellen zu vereinheitlichen. Er wird von den meisten **BIM!** (BIM!) Softwarelösungen verwendet, welche ihre Modellierungen als .ifc Datei exportieren können. Der IFC Standard besteht aus Objekten, Attributen und Beziehungen. Die Objekte sind hierarchisch aufgebaut. Jedes Objekt startet mit der Klasse *IfcRoot* und erbt weitere Attributen von allgemeineren Basisklassen. Das Beispiel zeigt die Hierarchie der Klasse *IfcWall* auf:

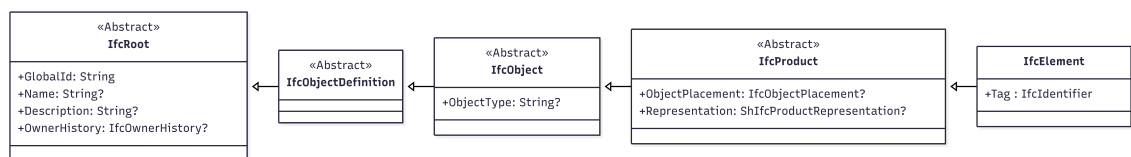


Abbildung 2.1: Hierarchischer Aufbau der IFC-Klassen.

Die Objekte werden mittels Beziehung in Verbindung gebracht. Zum Beispiel hängt ein Gebäude (*IfcBuilding*) mittels der Relation *IfcRelAggregates* an einem Bauplatz (*IfcSite*). Als visuelles Beispiel soll folgende Grafik dienen:

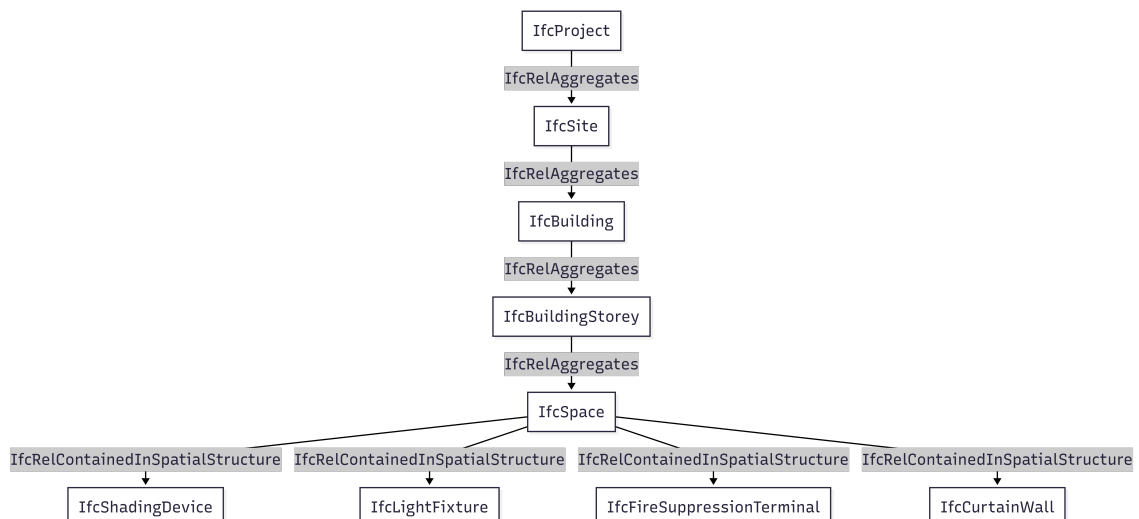


Abbildung 2.2: Struktureller Aufbau der IFC-Klassen.

2.2 Graphdatenbanken

Graphdatenbanken organisieren ihre Daten, wie der Name schon vermuten lässt, in Graphen. Das bedeutet Objekte werden als Knoten dargestellt, verbunden durch Kanten. Der große Vorteil hierbei ist die native Darstellung von Beziehungen zwischen den Objekten. Dies wird insbesondere wichtig bei Datenmengen, bei welchen die Beziehungen zueinander ein elementarer Bestandteil der Datenerhebung darstellt. Bei den gängigsten Graphdatenbanken handelt es sich um sogenannte Labeled Property Graphs. Hierbei können nicht nur Knoten, sondern auch Kanten mit Labels und Eigenschaften versehen werden. Dieser Ansatz

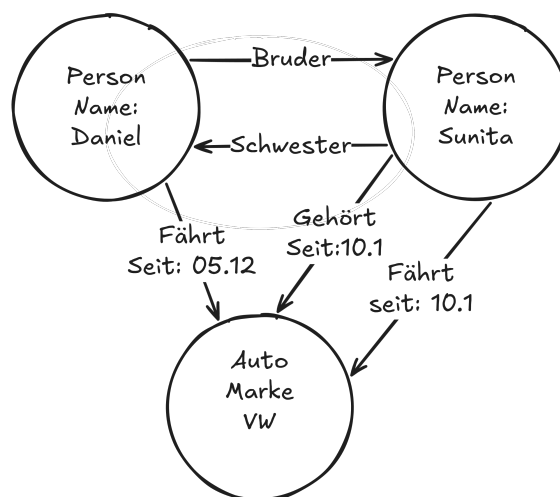


Abbildung 2.3: Labeled Property Graph

bietet eine hohe Flexibilität gegenüber relationalen Datenbanken, da hier der Datensatz logisch miteinander verknüpft werden kann und aufwändige JOIN Operationen vermieden werden können (?). Graphdatenbanken erfreuen sich vor allem großer Beliebtheit bei stark vernetzten Daten, wie Social Media, Betrugserkennung oder bei Lieferketten (?), da hierbei der Analyse der Verbindungen im Vordergrund steht.

2.2.1 neo4j

Neo4j ist eine **ACID (Atomicity, Consistency, Isolation, Durability)** konforme Graphdatenbank auf der Grundlage von Labeled Property Graphs. Sie wurde erstmalig 2007 von Emil Eifrem veröffentlicht und avancierte zu einer der weltweit führenden Graphdatenbanken. Ein zentrales Merkmal von Neo4j ist ihre Abfragesprache Cypher. Sie ist eine SQL ähnliche deklarative Sprache, das bedeutet, dass im Fokus die Frage steht was gesucht wird. Dabei folgt sie folgendem Grundaufbau:

- Knoten werden in Klammern dargestellt (Node)
- Beziehungen werden als Pfeile zwischen den Knoten dargestellt – [:FOLGT] –>
- Eigenschaften selbst sind Schlüssel-Wert-Paare: {name: 'Max'}

Folgend ist als Beispiel eine Abfrage über offene Mängel an einer technischen Komponente.

```
1 MATCH (c:Component {id: 'Pumpe-01'}) -[:HAS_ISSUE]->(i:Issue)
2 WHERE i.status <> 'CLOSED'
3 RETURN c, i
```

Listing 1: Abfrage von offenen Mängeln an einer technischen Komponente

2.2.2 GraphQL

Als Alternative zu [Rest-APIs](#) wurde GraphQL 2015 von Meta entwickelt. Der Vorteil liegt in der Flexibilität und erhöhten Effizienz der Abfragesprache. Im Gegensatz zu REST kann mit GraphQL die Anfrage strukturiert werden, um somit nur die benötigten Daten zu erhalten, wodurch sogenanntes Overfetching verhindert werden kann. Zusammengefasst kann der Entwickler genau definieren, welche Daten er abfragen möchte. Um dies zu ermöglichen, nutzt GraphQL vier Schlüsselkonzepte:

- Schemen
- Resolver
- Mutationen
- Abfragen

Das Herzstück von GraphQL sind die Schemas. Sie definieren die Struktur der Daten, welche abgefragt werden können und werden in der stark typisierten Schemadefinitionssprache (SDL) geschrieben.

Während Schemas die Struktur der Daten definieren, werden durch Resolver beschrieben, wie oder woher diese Daten abgerufen werden. Jedes Feld in einem Schema ist mit einem Resolver verknüpft, welcher definiert, wie dieses Datenfeld abgefragt wird.

Abfragen, oder auch *Queries* genannt, ist eine Anfrage des Clients an den Server. Sie beschreibt, welche Daten der Client abrufen möchte. Anstatt wie bei REST gehen alle Anfragen nur gegen eine GraphQL-Schnittstelle. Von dort an validiert der Server anhand der Schemas was angefragt wurde und bearbeitet sie. Für die Veränderung von Daten

sind Mutationen verantwortlich. Diese Vorgänge 'ersetzen', die POST-, PUT-, PATCH und DELETE Operatoren, welche von REST bekannt sind. Analog zu den Abfragen werden auch Mutationen in Schemas validiert. (?)

2.3 React

React ist eine von Meta entwickelte Javascript Bibliothek um Webseiten und [Single-Page-Application \(SPA\)s](#) zu entwickeln. Dabei fundiert React auf einen komponentenorientierten Ansatz, bei dem die Oberfläche in mehrere, wiederverwendbare Bauteile unterteilt wird. Dafür führte React die Syntaxerweiterung JSX ein, die Javascript, HTML und CSS in einer Datei vereint. React wurde seit seiner Einführung im Jahr 2013 zur populärsten Frontend-Bibliothek und hat sich als maßgebliche Methode für die Entwicklung moderner, interaktiver Benutzeroberflächen etabliert.

2.4 Microfrontends

Analog zu dem Konzept der Microservices Serverseitens, zerteilen Microfrontends das Frontend in mehrere, unabhängige Komponenten. Die grundlegende Idee dahinter ist es ein Problem in Teilprobleme zu zerteilen und getrennt zu bearbeiten. Neben diesem Vorteil bietet dieser Ansatz auch eine bessere Skalierbarkeit, Robustheit und Flexibilität. Vor allem unter Last können diese separat hochskaliert werden oder bei Fehlern fällt nicht immer zwangsläufig das ganze System aus, sondern nur das betroffene Microfrontend. Durch die modulare Architektur können auch die Technologien der Microfrontends angepasst werden. Somit ist ein Entwickler nicht auf eine Technologie beschränkt, sondern kann den Anforderungen entsprechend die Beste für die benötigten Anforderungen wählen. Neben diesen Vorteilen existieren auch Nachteile, welche Microfrontends mitbringen. Ein großer Nachteil ist die erschwerte Kommunikation zwischen den Komponenten und mögliche Synchronisationsprobleme und natürlich der erhöhte Koordinationsaufwand.

2.4.1 Module Federation

Module Federation ist eine von Zack Jackson erfundene Architektur, welche 2020 mit dem Release von [Webpack 5](#) als Feature veröffentlicht wurde. Es bietet die Möglichkeit Code

externer Anwendungen zur Laufzeit zu laden und somit monolithische Anwendungen in Microfrontends zu unterteilen.

3 Graphbasiertes Datenmodell und Schnittstellendesign

Der IFC Standard ist ein stark vernetztes und hierarchisches Datenmodell, bei welchem die Beziehungen selbst oft eine sehr wichtige Rolle einnehmen. Während relationale Datenbanken die Daten in starre Tabellen zwingen und Beziehungen aufwändig über rechenintensive JOIN Operatoren modelliert werden müssen, können Graphdatenbanken diese effizient darstellen (?). Somit könne Datenpunkte mit deren zugehörigen Beziehungen gezielt abgefragt werden. Zudem kann aufgrund der graphbasierten Struktur der IFC-Standard realitätsnah und intuitiv dargestellt werden.

3.1 Graphbasierte Domänenmodellierung für IFC-Daten

Das Ziel der Domänenmodellierung ist es, mit Hilfe der Graphdatenbank eine möglichst realitätsnahe und intuitive Abbildung der bereits bestehenden IFC Dateien zu erschaffen. Einen sogenannten *digitalen Zwilling*. IFC Dateien nutzen das [STEP](#) Dateiformat, um ihren Inhalt zu beschreiben.

```

1 ISO-10303-21;
2 HEADER;
3 FILE_DESCRIPTION(('ViewDefinition [CoordinationView_V2.0]')
4   , '2;1');
5 FILE_NAME('Bautagebuch_Modell.ifc', '2026-06-02T22:15:00', ('
6   Architekt Max'), ('Ingenieurbuero X'), 'IFC-Exporter v1.0', 'CAD
7   -BIM-Software 2026', _ );
8 FILE_SCHEMA(('IFC4'));
9 ENDSEC;
10 DATA;
11 10= IFCPROJECT('0Yn$b7pX187R3W84z_uXy1', #15, 'Projekt
12   Grossbaustelle IBN', 'Beschreibung des Grossprojekts', $, $, $
13   , (#40), #30);
14 15= IFCOWNERHISTORY(#20, #25, $, .ADDED., $, $, $, 1717366500);
15 20= IFCPERSONANDORGANIZATION(#21, #22, $);
16 21= IFCPERSON('ID_01', 'Mustermann', 'Max', $, $, $, $, $);
17 22= IFCORGANIZATION('ORG_01', 'Bauunternehmung GmbH', $, $, $);
18 25= IFCAPPLICATION(#22, 'v1.0', 'BIM-Software', 'App_ID');

```

Listing 2: Beispielhafter IFC-Datenauszug im STEP-Dateiformat (ISO 10303-21).

Man kann dieses Schema bereits als eine Art Graph verstehen. Die Objekte selber können als Knoten mit ihren Attributen als Daten und die Verweise der Objekte (z.B. #10 verweist auf #15) als Kanten, also deren Beziehungen interpretiert werden.

Durch ein graphbasiertes Domänenmodell lässt sich der IFC Standard einfach, nahezu verlustfrei (?) und realitätsnah abbilden (?). Um dies zu erreichen werden konkrete IFC Klassen wie *IfcWall* oder *IfcSpace* als Knoten dargestellt und Beziehungen wie *IfcRelAggregates* als Kanten. Die Attribute dieser Klassen werden in den Objekten analog zum IFC Standard, als Eigenschaften gespeichert. Hier zeigt sich der Vorteil von auf Labeled Property Graphs basierenden Graphdatenbanken, wie zum Beispiel Neo4j. Man kann die Knoten und Kanten mit sogenannten *Labels* versehen, also Bezeichnungen. Dies ermöglicht es, diese als ihre äquivalenten IFC Klassen oder Beziehung zu kennzeichnen. Mit dieser Methode lässt sich ein objektorientiertes Domänenmodell erstellen, ähnlich der objektorientierten Softwarearchitektur (?).

3.2 GraphQL als Vermittlungsinstanz

Für eine effiziente und IFC konforme Kommunikation wird als Vermittlungsschicht GraphQL zwischen den Clients und der Graphdatenbank implementiert. Diese Middleware hilft bei der Lösung zentraler Herausforderungen, indem sie Lösungen zu Typsicherheit und detailliertem Datenabruf bereitstellt. Während Knoten in Graphdatenbanken beliebig definiert werden können und somit eine konsequente Einhaltung des IFC-Standards erschwert, können mittels GraphQL Schemata definiert werden, um genau jenes zu verhindern (?). Folgend ist exemplarisch ein Schemata zu der IFC Klasse *Object* und dessen Superklassen zu sehen:

Even-
tuell in
eigenes
Kapitel
schrei-
ben

nahezu
ver-
lustfrei
siehe
Giehl
4.3 (evtl.
erwäh-
nung
eines
sekun-
dargra-
phens
siehe
Koray
Inal sei-
te 40)

Graphik
mit
zwei
objek-
ten und
einer
bezie-
hung

Aus-
zug der
GRaph-
daten-
bank

```
1 interface IfcRoot {
2     globalId: ID!
3     name: String
4     description: String
5 }
6
7 interface IfcObjectDefinition implements IfcRoot {
8     globalId: ID!
9     name: String
10    description: String
11 }
12
13 interface IfcObject implements IfcRoot & IfcObjectDefinition {
14     globalId: ID!
15     name: String
16     description: String
17     objectType: String
18 }
```

Listing 3: GraphQL-Schema für IFC-Kernausschnitte

Durch das Definieren dieser Schemata kann mit der Nutzung von GraphQL eine strikte Typvalidierung durchgesetzt werden. Dies forciert IFC konforme Datenmodelle innerhalb der Graphdatenbank, indem GraphQL serverseitig eingehende Anfragen gegen die Schemata validiert (?). Gleichzeitig können Änderungen der Graphdatenbank mittels Mutationen gesteuert werden, sodass diese als Validierungsinstanz fungieren, um die IFC Konformität zu gewährleisten. Da jede Mutation ein strikt typisiertes Schemata bestehen muss, verhindert dies ein unkontrolliertes einbringen nicht standardisierter Objekte. Ein weiterer Vorteil liegt in der Optimierung von Datenabfragen. Während klassische REST APIs zu Over- oder Underfetching neigen, eliminiert GraphQL diese vollständig. Die Clients definieren durch ihre Queries selbstständig welche Daten sie benötigen. Diese Queries werden von Resolvem (siehe) verarbeitet, indem sie die Anfragen in die Abfragesprache Cypher übersetzen, um effizient die Daten aus der Graphdatenbank abzurufen.

3.3 GraphQL als einheitliche API

In einer Add-On fähigen Softwarearchitektur, in der ggf. unterschiedliche Entwickler arbeiten, ist eine flexible API von großem Vorteil. Während REST-APIs auf starre vordefinierte Endpunkte setzen, basiert GraphQL auf clientgesteuerte Abfragen. Somit können Entwickler benötigte Schnittstellen flexibel und eigenständig über die bestehenden Schemata abbilden. Zudem können Clients mittels der sogenannten *Schema Introspection*, die vorhandenen Schemata abfragen. Durch diese Transparenz entfallen sonst notwendige Dokumentationen, wodurch die Einstiegshürden für neue Entwickler drastisch gesenkt werden und das Erstellen neuer Add-Ons auf Basis des Domänenmodells vereinfacht wird.

Neben diesen Vorteilen bietet das Verwenden von GraphQL als Middleware eine wesentlich zugänglichere Abstraktionsebene als gängige Abfragesprachen für Graphdatenbanken wie Cypher oder SPARQL.

4 Modulare Softwarearchitektur mittels Module Federation

5 Verknüpfung der Microfrontends und des Servers

6 Zusammenfassung und Fazit

7 Ausblick

Erklärung

Der Verfasser erklärt, dass er die vorliegende Arbeit selbständig, ohne fremde Hilfe und ohne Benutzung anderer als die angegebenen Hilfsmittel angefertigt hat. Die aus fremden Quellen (einschließlich elektronischer Quellen) direkt oder indirekt übernommenen Gedanken sind ausnahmslos als solche kenntlich gemacht. Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer anderen Prüfung noch nicht vorgelegt worden. Zudem bestätigt der Verfasser, dass er den Lehrstuhlleitfaden in der jeweiligen geltenden Fassung gelesen und verstanden hat und sich daher über die gestellten Anforderungen und Bewertungsmaßstäbe im Klaren ist.

Datum, Ort

Jan Händl

Anhang

Literatur

Denis Feth, Thomas Jeswein, and Stefanie Ludborzs. Studie: Stand der digitalisierung in der baubranche. *Fraunhofer IESE*, 12, 2023.

BBSR. Beitrag der digitalisierung zur produktivität in der baubranche. Bbsr-online-publikation, Bundesamt für Bauwesen und Raumordnung (BBR), Bonn, 10 2019.

Ali Ismail, Ahmed Nahar, and Raimar Scherer. Application of graph databases and graph theory concepts for advanced analysing of bim models based on ifc standard. *Proceedings of EGICE*, 161, 2017.

Google Cloud. Was ist eine graphdatenbank?

Udo Seidel. Beziehungsfragen: Einstieg in graph-datenbanken.

IBM. Was ist graphql?, 2024. Think Topics.

Objectbay. Microservices und das web: Integration von microservices in die ui, 2024. Blogpost zur Frontend-Integration von Microservices.

Hamza Alkhatib, Svetlana Olbina, and Raja R. A. Issa. A graph database and query approach to ifc data management. *Journal of Information Technology in Construction (ITcon)*, 26:89–112, 2021.